

Tecnológico de Monterrey

Campus Santa Fe

Declaración de IA

Julian Berges
Daniel Armas
Guillermo Gonzalez
Emilio Torres Castillo

23/10/2026

Emilio Torres

+ Apoyo para la creación de función para soloRoles() en backend/index.js:

- Herramienta: Claude
- Prompts:
  - "Mira pero es que para checar el rol en cada acción estaba pensando en que el front mandara el rol que manda el json del log in pero no se como hacer eso en el frontend"
  - "Vamos con los roles"
  - "así está bien la función soloRoles a mi forma:"

```
// Middleware que valida el token JWT y verifica los permisos del usuario según su rol.
function soloRoles(permitidos){
    return async (req, res, next)=>{
        try{
            let token=req.get("Authentication");
            let verifiedToken=jwt.verify(token, process.env.JWT_SECRET);
            const [usuarios]=await db.query(
                "SELECT id_usuario, usuario, rol, id_region, id_ebdi FROM usuarios WHERE usuario = ? AND activo = 1",
                [verifiedToken.Usuario]
            );
            if(usuarios.length === 0){
                return res.sendStatus(401);
            }
            req.usuario=usuarios[0];
            if(permitidos.includes(req.usuario.rol)){
                return next();
            }
            return res.status(403).json({message: "Tu rol no puede hacer esto"});
        }
        catch(error){
            if(error instanceof jwt.JsonWebTokenError){
                return res.sendStatus(401);
            }
            console.error("Error verificando permisos:", error);
            return res.status(500).json({message: "No se pudieron verificar los permisos"});
        }
    }
}
```

+ Apoyo para hacer fetch con el hook de useGetList() para poder obtener matrículas de las infantes y ponerlo como un listado (SelectInput) al hacer registros de mediciones de una EDBI y que solo mostrara las matrículas que pertenecen a esa instancia en particular.

- Herramienta: VsCode Copilot
- Prompts:
  - "Reajusté la implementación porque no la entendía y siento que no lo iba a poder explicar en el proyecto. Te explico lo que busco hacer.

    En el backend: ya hice la función que nos regresa un listado de matrículas por id_ebdi (solo muestra los niños de esa instancia), y el GET /matriculaInfantes pide el token y usuario al front para poder usarlo en el WHERE clause. Dame retroalimentación allí y explícame si estamos bien.

    En el front, lo único que necesita son peso, talla, talla_blanco_familiar y perímetro cefálico por escrito, y un tipo de SelectInput que llama a /matriculaInfantes.

Dame el código que cambiarías para la parte del front por aquí en nuestro chat y dame retroalimentación de lo que hice en el backend”

```
interface MatriculaInfante {
    id: number;
    matricula: number;
}
```

```
const FormularioCrearMedicion = () => {
    const { data: infantes = [], isPending, error } =
        useGetList<MatriculaInfante>("matriculaInfantes", {
            pagination: { page: 1, perPage: 1000 },
            sort: { field: "matricula", order: "ASC" },
        });

    if (isPending) return <p>Cargando matrículas...</p>;
    if (error) return <p>No se pudieron cargar las matrículas.</p>;

    const choices = infantes.map((infante) => ({
        id: infante.matricula,
        matricula: infante.matricula,
    }));
```

```
export const crearMedicion = () => <FormularioCrearMedicion />;
```

+ Apoyo para debuggear error en el rol Regional, ya que se usaba el r.id_region pero no se había hecho JOIN para la tabla de Regiones
- Herramienta: Vscode copilot
- Prompts:
    - “Checa error en los roles porque se supone que un usuario con rol = "regional" debería tener su WHERE clause usando id_region, sin embargo, es el único rol que falla. El padre, estancia y admin funciona. Ya corroboré con queries en mysql: SELECT
    - p.id_infante AS id,
    - p.matricula,
    - p.nombre,
    - p.paterno,
    - p.materno,
    - p.genero,
    - p.fnac,

- e.nombre AS ebdi,
- s.nombre AS estado
- FROM padron AS p
- INNER JOIN ebdis AS e
- ON e.ur = p.ur
- AND e.ebdi_num = p.ebdi
- INNER JOIN estados AS s
- ON s.id_estado = e.id_estado
- INNER JOIN regiones AS r
- ON r.id_region = s.id_region
- WHERE p.activo = 1 AND r.id_region = 1; “

```
const [mediciones] = await db.query(
  "SELECT m.id_medicion AS id, m.id_infante, m.fecha, m.peso, m.talla, m.imc, m.talla_blanco_familiar, m.perimetro_cefalico, " +
  "TIMESTAMPDIFF(MONTH, p.fnac, m.fecha) AS edad_meses, p.nombre AS infante, e.nombre AS nombre_ebdi, s.nombre AS estado" +
  " FROM mediciones AS m INNER JOIN padron AS p ON p.id_infante = m.id_infante" +
  " INNER JOIN ebdis AS e ON e.ur = p.ur AND e.ebdi_num = p.ebdi" +
  " INNER JOIN estados AS s ON s.id_estado = e.id_estado" +
  " INNER JOIN regiones AS r ON r.id_region = s.id_region" +
  " WHERE p.activo = 1" + alc.sql+
  " ORDER BY " + columna + " " + direccion +
  " LIMIT ? OFFSET ?",
  [...alc.params, cuantos, inicio]);
```

+ Planeación de esquema nuevo para las tablas requeridas para la base de datos inicial del padrón
+ Herramienta: Claude
+ Prompts:
  - Es una conversación larga para comprender el esquema propuesto pero el resumen que dió la IA fue el siguiente: “Entre el 13 y el 17 de octubre, el alumno consultó a Claude para diseñar las tablas que le faltaban a la base del socio formador (que solo traía padron). Preguntó por la llave de ebdis (un id numérico además del par ur y ebdi_num), por las columnas de usuarios (usuario y rol, región y EBDI para limitar lo que ve cada rol, y la columna activo), por la tabla puente padres_infantes y por la tabla mediciones (incluida la elección de FLOAT). También pidió un seed.sql con datos de prueba, preguntó por el significado de id_benef_infante y de la columna usuario de padron, y pidió ayuda con el error 1452 al insertar mediciones. Claude explicó cada decisión, propuso la tabla de mediciones y el seed, y diagnosticó el error. El alumno escribió el archivo extraPadron.sql y tomó las decisiones finales.”

```sql
USE padronISSSTE;

CREATE TABLE regiones(  -- Tabla regiones para poder ligar ebdis o usuarios a cierta region
    id_region TINYINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  -- Esta información no se encuentra en la base de datos por eso la agregamos
    nombre VARCHAR(10) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE estados( -- Nuevamente, la base de datos do cuenta con un identificador claro de estaodo
    id_estado TINYINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  -- Agregamos esta tabla para poder referenciar el nombre del estaod donde se encuentra una EBDI
    nombre VARCHAR(40) NOT NULL,
    id_region TINYINT UNSIGNED NOT NULL,
    FOREIGN KEY (id_region) REFERENCES regiones(id_region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE ebdis(  -- Creamos esta tabla para poder referenciar mas datos de las EBDI como su tipo, nombre y poder ligarla a usuarios y/o lugraes
    id_ebdi INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  -- Esta llave primaria la creamos aparte del par (ur, ebdi_num) para poder juntar con las demás tablas con un id numerico
    ur VARCHAR(3) CHARACTER SET latin1 COLLATE latin1_swedish_ci NOT NULL,  -- y que no dependa de un par de varchar()
    ebdi_num INT NOT NULL,
    id_estado TINYINT UNSIGNED NOT NULL,
    nombre VARCHAR(80) NOT NULL,
    tipo ENUM('PROPIA','CONTRATADA','OSC') NOT NULL DEFAULT 'PROPIA',
    UNIQUE KEY (ur, ebdi_num),  -- Esta unique key la usamos como la "llave primaria" para poder referenciar la tabla Padron del socioformador
    FOREIGN KEY (id_estado) REFERENCES estados(id_estado)  -- Parece ser que el id ur y ebdi en padron indican el identificador de una ebdi por eso la ocupamos para unir ambas tablas
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE usuarios(  -- Creamos una tabla usuarios porque la base de datos no contaba con registros para los usuarios ni contraseñas, solo tenia la matricula del infante
    id_usuario INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  -- lo cual no seria seguro para ocupar como Log in
    usuario VARCHAR(30) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    rol ENUM('admin','regional','estancia','padre') NOT NULL, -- Basada en esta columana con WHERE podemos filtrar los permisos
    id_region TINYINT UNSIGNED NULL,  -- Necesitamos referenciar la region para roles de administrador regional, usariamos WHERE
    id_ebdi INT UNSIGNED NULL,     -- Para los roles de instance necesitamos referencias solo la EBDI que corresponde
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (id_region) REFERENCES regiones(id_region),
    FOREIGN KEY (id_ebdi) REFERENCES ebdis(id_ebdi)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE padres_infantes(   -- Tabla puente entre usuarios y padron para la relacion M:M (padres con varios hijos)
    id_usuario INT UNSIGNED NOT NULL, -- Esto implica que un usuario puede ver informacion de varios hijos
    id_infante INT NOT NULL,          -- y también que padre o madre pueden tener 2 usuarios distintos por ende repitiendo los mismos hijos por usuario
    PRIMARY KEY (id_usuario, id_infante), -- el par padre e hijo es único (padre1, hijo2)
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_infante) REFERENCES padron(id_infante)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE mediciones(
    id_medicion INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    id_infante INT NOT NULL,                -- mismo tipo que padron.id_infante
    fecha DATE NOT NULL,
    peso FLOAT NOT NULL,              -- kg
    talla FLOAT NOT NULL,             -- cm
    talla_blanco_familiar FLOAT NOT NULL,
    perimetro_cefalico FLOAT NOT NULL, -- cm
    id_usuario INT UNSIGNED NOT NULL,         -- quién la capturó
    FOREIGN KEY (id_infante) REFERENCES padron(id_infante),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Daniel Armas

+ Apoyo para adaptar a TypeScript el control de acceso por rol que te da el Tutorial de React-Admin (canAccess), en el AuthProvider.ts
+ Herramienta: Claude
+ Prompts:
- “Necesito que cada rol vea solo ciertos botones y menús: el admin todo, la estancia todo menos usuarios, y el regional y el padre solo ver. Escribí el canAccess siguiendo el ejemplo del tutorial de React Admin, pero me marca muchos errores y no sé por qué:

```
const accessControlStrategies={

  admin: ({resource, action})=>{

    return true;

  },

  estancia: ({resource, action})=>{

    return resource !== "usuarios";

  },
```

```
  regional: ({resource, action})=>{
      return resource !== "usarios" && (action=="list" || action =="show");
  },
  padre: ({resource, action})=>{
      return resource !== "usarios" && (action=="list" || action =="show");
  },
};
”
```

```
import { AuthProvider } from "react-admin";
type CanAccessParams = {
    action: string;
    resource: string;
};
const accessControlStrategies: Record<string, (params: CanAccessParams)=>boolean>={
    admin: ()=>{
        return true;
    },
    estancia: ({resource})=>{
        return resource !== "usuarios";
    },
    regional: ({resource, action})=>{
        return resource !== "usuarios" && (action=="list" || action =="show");
    },
    padre: ({resource, action})=>{
        return resource !== "usuarios" && (action=="list" || action =="show");
    },
};
```

+Apoyo para encontrar por qué no funcionaba borrar un infante
+Herramienta: Claude

+Prompt: “Hice una función para borrar un infante y la ruta delete, pero cuando le doy borrar en React Admin no lo encuentra. Revisa mi código y dime qué está mal:
async function deleteInfante(req, res){

let id = Number(req.params.id);

const[ninos]=await db.query("SELECT " + columnasInfantes + " FROM padron WHERE id_infante = ?", [id]);

await db.query("DELETE FROM padron WHERE id_infante = ?", [id]);

res.json(ninos[0]);

}, app.delete("/infantes/:id,", async (req, res)=>{

await deleteInfante(req, res);

});”

```
// borrar a un infante
async function deleteInfante(req, res){
    let id = Number(req.params.id);
    await db.query("DELETE FROM padron WHERE id_infante = ?", [id]);
    res.json({success: true});
}
```

+Apoyo para corregir errores de escritura en las funciones de usuarios
+Herramienta: Claude

+Propm**t:** “Hice las funciones para ver y editar un usuario, pero cuando el admin abre un usuario me dice que no existe. Revisa mi código y dime qué está mal: async function getOneUsuario(req, res){

let id= Number(req.params.id);

let consulta="SELECT id_usuaario AS id, usuario, rol, id_region, id_ebdi, activo FROM usuarios WHERE id_usuario = ?";

const [usuario]=await db.query(consulta, [id]);

res.json(usuario[0]);

}

async function updateUsuario(req, res) {

let id=Number(req.params.id);

let datos=req.body;

let consulta="UPDATE usuarios SET usuarios = ?, rol=?, id_region=?, id_ebdi=? WHERE id_usuario=?";

await db.query(consulta, [datos.usuario, datos.rol, datos.id_region, datos.id_ebdi, id]);

await getOneUsuario(req, res);

}

app.get("/usuario/:id", soloRoles(["admin"]), async (req, res)=>{

await getOneUsuario(req, res);

});”

```
// un solo usuario
async function getOneUsuario(req, res){
    let id= Number(req.params.id);
    let consulta="SELECT id_usuario AS id, usuario, rol, id_region, id_ebdi, activo FROM usuarios WHERE id_usuario = ?";
    const [usuario]=await db.query(consulta, [id]);
    res.json(usuario[0]);
}
```

```
// editar a un usuario
async function updateUsuario(req, res) {
    let id=Number(req.params.id);
    let datos=req.body;
    let consulta="UPDATE usuarios SET usuario = ?, rol=?, id_region=?, id_ebdi=? WHERE id_usuario=?";
    await db.query(consulta, [datos.usuario, datos.rol, datos.id_region, datos.id_ebdi, id]);
    await getOneUsuario(req, res);
}
```

```
// endpoint get un usuario
app.get("/usuarios/:id", soloRoles(["admin"]), async (req, res)=>{
    await getOneUsuario(req, res);
});
```

+Apoyo para agregar un link de registro en la pantalla de login (MyLogin)
+Herramienta: Claude

+Propm**t:** “Para el login de React-Admin, ¿No puedo hacerlo con un hook el cambio de pantalla para otra, en el tutorial de React Admin no viene alguna forma de hacerlo?

Claude me explicó que la pantalla de login la dibuja React Admin por dentro, por eso un hook no alcanza. Me mostró la sección “Customizing The Login Component” del Tutorial se hace un componente propio con Login y LoginForm, se le agrega el link y se le pasa a Admin con loginPage.

```
const MyLogin = () => (
    <Login>
        <LoginForm />
        <Box textAlign="center" mb={1}>
            <Link component={LinkBase} to="/registrarse">
                Registrarse
            </Link>
        </Box>
    </Login>
);
```

Julian Berges

+ Apoyo en el debug de Admin_inicio.tsx con chat gpt
- Chat GPT
- Prompt -

fix , import { Card, CardContent, CardHeader } from "@mui/material";
import {BooleanField,CreateButton,Datagrid,EditButton,ListBase,TextField,CreateButton} from "react-admin";

```
export const AdminInicio = () => (
  <Card>
    <CardHeader
    title="Bienvenido, administrador"
    subheader='En esta sección puedes consultar todos los perfiles de la plataforma. En "Infantes" puedes ver, crear, editar y eliminar los registros de los niños. En "Gráficas" puedes consultar su crecimiento y desarrollo.'
    action={<CreateButton resource="usuarios" label="Crear perfil">/}
    />
    <CardContent>

      <ListBase resource="usuarios" sort={{ field: "id", order: "ASC" }}>
        <Datagrid bulkActionButtons={false} rowClick="edit">
          <TextField source="id" label="ID" />
          <TextField source="usuario" label="Usuario" />
          <TextField source="rol" label="Rol" />
          <TextField source="region" label="Región" />
          <TextField source="ebdi" label="EBDI" />
          <BooleanField source="activo" label="Activo" />
          <EditButton />
        </Datagrid>
      </ListBase>
    </CardContent>
  </Card>
);
```

El resultado fue el actual archivo de Admin_inicio.tsx

+ La creación de los usuarios en seed.sql
- Chat gpt
- Prompt -
    - create a user from the terminal into the db

```sql
INSERT INTO usuarios
(usuario, password_hash, rol, activo)
VALUES
('padre001', 'PASTE_YOUR_GENERATED_HASH', 'padre', 1);
```

- **segundo prompt**
  - argon2 hash of password = 1

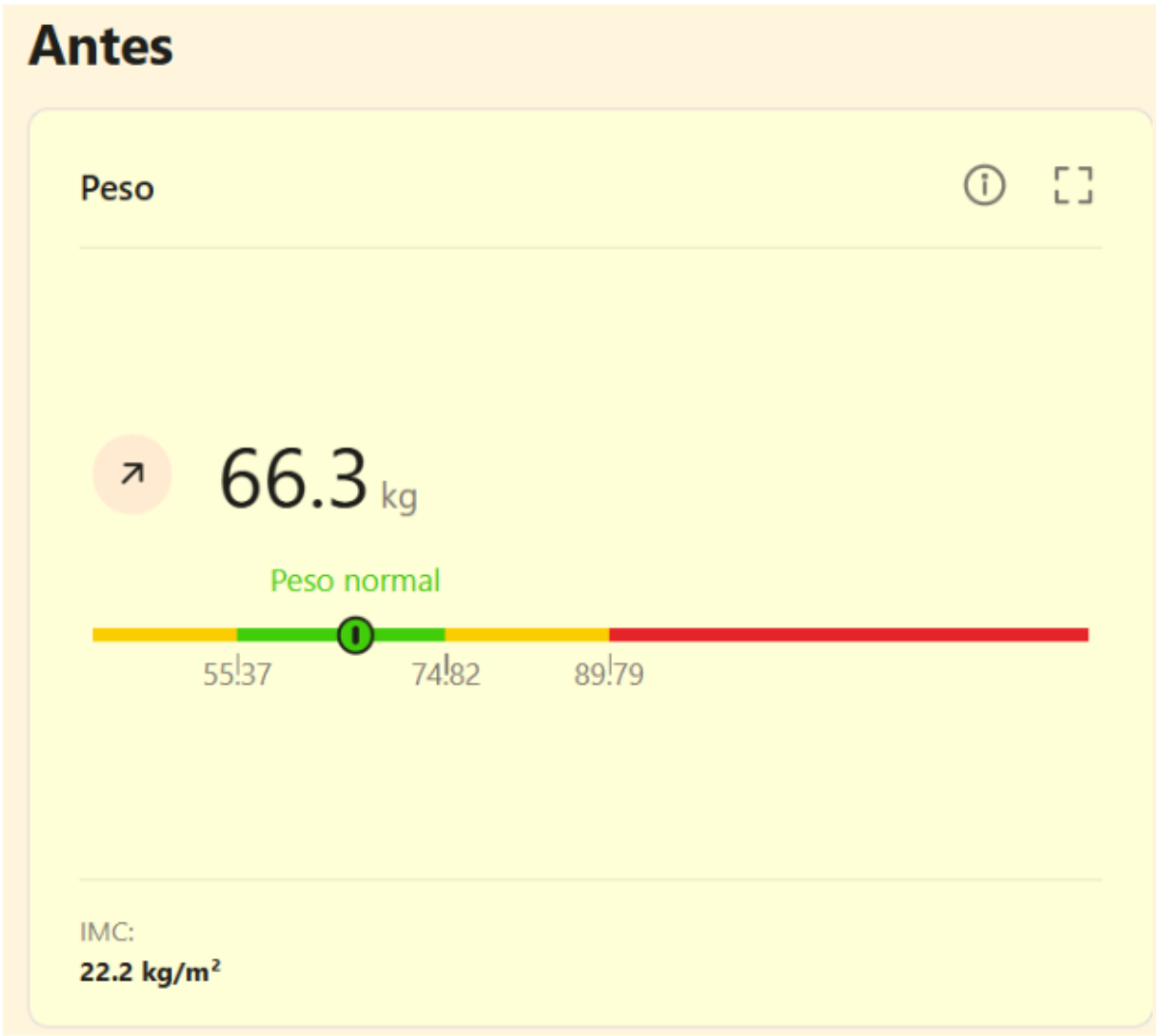
```sql
INSERT INTO usuarios (usuario, password_hash, rol, activo)
VALUES (
    'padre001',
    '$argon2id$v=19$m=65536,t=3,p=1$vKPCrH2GfEIOOamUVEpw8g$TntaUz3NzyhXVgGie5Qvs6XK1JHo2gPvIwvXeKtkQlw',
    'padre',
    1
);
```

```sql
INSERT INTO usuarios (usuario, password_hash, rol, activo)
VALUES (
    'padre1',
    '$argon2id$v=19$m=65536,t=3,p=1$vKPCrH2GfEIOOamUVEpw8g$TntaUz3NzyhXVgGie5Qvs6XK1JHo2gPvIwvXeKtkQlw', -- 1 PW
    'padre',
    1
);
```

Guillermo Gonzalez

- Uso de Claude para generar código
- Uso de Claude para revisiones de errores

Herramienta: Claude
Prompt: “Como puedo crear una gráfica como la que se puede observar a continuación en una página web creada con react admin”



Resultado:

```
import { Card, CardContent, Box, Typography, Divider } from "@mui/material";
import NorthEastIcon from "@mui/icons-material/NorthEast";
import SouthEastIcon from "@mui/icons-material/SouthEast";
import EastIcon from "@mui/icons-material/East";

const COLORES = {
    amarillo: "#f5c400",
    verde: "#3cc619",
    rojo: "#ee2a2a",
};

const TRAMOS = [
    { color: COLORES.amarillo, nombre: "Bajo peso" },
    { color: COLORES.verde, nombre: "Peso saludable" },
    { color: COLORES.amarillo, nombre: "Sobrepeso" },
    { color: COLORES.rojo, nombre: "Obesidad" },
];
```

```jsx
export const IMCGraph = ({
    titulo = "Peso",
    valor,
    unidad = "kg",
    anterior,
    umbrales,
    imc,
    nombres,
}) => {
    //Recibe el objeto que envia el backend
    const [b1, b2, b3] = umbrales;

    /* Rango visible de la barra: un poco antes de b1 y bastante después de b3
    Se utiliza el min y max para que los imc que sobrepasen los limites
    de la barra no se salgan de ella y se vean mal*/
    const min = Math.min(b1 - (b2 - b1) * 0.5, imc);
    const max = Math.max(b3 + (b3 - b2) * 1.5, imc);
    /*Esta funcion se usa para identificar donde se encuentra el valor
    dentro de la barra, se utiliza mas tarde para colocar
    cosas dentro de la barra*/
    const pct = (v) => ((v - min) / (max - min)) * 100;
```

```jsx
    let indice = 0;
    if (imc >= b3) indice = 3;
    else if (imc >= b2) indice = 2;
    else if (imc >= b1) indice = 1;

    const tramo = TRAMOS[indice];
    const etiqueta = nombres[indice];
    /*el objeto en nombres contiene los nombres de los tramos, aqui se selecciona
    cual es el que aplica para este IMC*/

    // Los 4 segmentos con su ancho
    const limites = [min, b1, b2, b3, max];
    const segmentos = TRAMOS.map((t, i) => ({
        color: t.color,
        ancho: pct(limites[i + 1]) - pct(limites[i]),
        //aqui se usa la funcion pct para calcular el ancho de cada segmento de l
    }));

    // Flecha de tendencia, si hay un valor anterior, se compara con el actual
    let Flecha = EastIcon;
    if (anterior != null && imc > anterior) Flecha = NorthEastIcon;
    if (anterior != null && imc < anterior) Flecha = SouthEastIcon;
```

```jsx
    return (
        <Card sx={{ width: 480, maxWidth: "100%", borderRadius: 3 }}>
            <CardContent sx={{ p: 3 }}>
                <Typography variant="subtitle1" fontWeight={600}>
                    {titulo}
                </Typography>
                <Divider sx={{ my: 1.5 }} />

                {/* Valor grande + flecha */}
                <Box sx={{ display: "flex", alignItems: "center", gap: 2, mt: 4 }}>
                    <Box
                        sx={{
                            width: 44,
                            height: 44,
                            borderRadius: "50%",
                            bgcolor: "action.hover",
                            display: "flex",
                            alignItems: "center",
                            justifyContent: "center",
                        }}
                    >
                        <Flecha fontSize="small" />
```

```jsx
                        {valor}
                        <Typography variant="body1" component="span" color="text.
                            {unidad}
                        </Typography>
                    </Typography>
                </Box>

                {/* Etiqueta sobre la barra, alineada con el marcador */}
                <Box sx={{ position: "relative", mt: 3, height: 22 }}>
                    <Typography
                        variant="body2"
                        sx={{
                            position: "absolute",
                            left: `${pct(imc)}%`,
                            transform: "translateX(-50%)",
                            color: tramo.color,
                            fontWeight: 500,
                            whiteSpace: "nowrap",
                        }}
                    >
                        {etiqueta}
                    </Typography>
                </Box>
```

```jsx
{/* Barra segmentada + marcador */}
<Box sx={{ position: "relative", mt: 0.5 }}>
    <Box sx={{ display: "flex", height: 6, borderRadius: 3, overf
        {segmentos.map((s, i) => (
            <Box key={i} sx={{ width: `${s.ancho}%`, bgcolor: s.c
        ))}
    </Box>
    <Box
        sx={{
            position: "absolute",
            top: "50%",
            left: `${pct(imc)}%`,
            width: 18,
            height: 18,
            transform: "translate(-50%, -50%)",
            borderRadius: "50%",
            bgcolor: tramo.color,
            border: "2px solid",
            borderColor: "text.primary",
        }}
    />
</Box>
```

```jsx
{/* Números bajo los límites */}
<Box sx={{ position: "relative", height: 24, mt: 1 }}>
    {[b1, b2, b3].map((b) => (
        <Typography
            key={b}
            variant="caption"
            color="text.secondary"
            sx={{
                position: "absolute",
                left: `${pct(b)}%`,
                transform: "translateX(-50%)",
            }}
        >
            {b}
        </Typography>
    ))}
</Box>
```

```
                {imc != null && (
                    <>
                        <Divider sx={{ mt: 4, mb: 1.5 }} />
                        <Typography variant="caption" color="text.secondary">
                            IMC:
                        </Typography>
                        <Typography variant="body2" fontWeight={700}>
                            {imc} kg/m²
                        </Typography>
                    </>
                )}
            </CardContent>
        </Card>
    );
};
```

Herramienta: Claude
Prompt: “Hazme un endpoint tomando en cuenta las siguientes tablas {Le pase los archivos de padrón y de extra padrón} Que recupere los datos de mediciones en el backend y los mande a la gráfica que me generaste previamente.
Resultado:

```
        const [filas] = await db.query(
            "SELECT p.id_infante, p.genero, m.peso, m.talla, m.fecha, m.imc," +
            " TIMESTAMPDIFF(MONTH, p.fnac, m.fecha) AS edad_meses" +
            " FROM padron AS p" +
            " INNER JOIN ebdis AS e ON e.ur = p.ur AND e.ebdi_num = p.ebdi" +
            " INNER JOIN estados AS s ON s.id_estado = e.id_estado" +
            " INNER JOIN mediciones AS m ON m.id_infante = p.id_infante" +     //
            " WHERE p.activo = 1 AND p.id_infante = ?" + alc.sql +
            " ORDER BY m.fecha DESC, m.id_medicion DESC LIMIT 2",
            [id, ...alc.params]
        );
        if(filas.length == 0){
            return res.status(404).json({message: "Sin mediciones para este infar
        }
        //Esta busqueda de sql recupera todos los datos necesarios para la grafic
        const actual = filas[0];
        const anterior = filas[1];
        //la idea de tomar dos mediciones es de Claude
```

```
        const umbrales = [Number(refs[0].p5), Number(refs[0].p85), Number(refs[0]

        res.json({
            id: id,
            valor: Number(actual.peso),
            unidad: "kg",
            anterior: anterior ? Number(anterior.peso) : null,
            umbrales: umbrales,
            imc: Number(actual.imc),
            edad_meses: actual.edad_meses,
            nombres: ["Bajo peso", "Peso saludable", "Sobrepeso", "Obesidad"],
        });
    }
    catch(err){
        console.error("Error en GET /infantes/:id/estado:", err);
        res.status(500).json({message: "No se pudo calcular el estado"});
    }
});
```

Herramienta: Claude
Prompt: “como hago que se imprima donde mismo que las otras dos graficas”
Resultado:

```
export const IMCIndividualGraphCard = ({ idInfante, nombre }) => {
  const { data, isPending, error } = useGetOne("infantes", { id: `${idInfante}/

  if (isPending) return <p>Cargando...</p>;
  if (error) {
      return (
          <Card sx={{ width: 600, maxWidth: "100%", borderRadius: 3 }}>
              <CardHeader title={`Peso - ${nombre}`} />
              <CardContent>{error.message ?? "No se pudo cargar el estado"}</Ca
          </Card>
      );
  }

  return (
      <IMCGraph
          titulo={`Peso - ${nombre}`}
          valor={data.valor}
          unidad={data.unidad}
          anterior={data.anterior}
          umbrales={data.umbrales}
          imc={data.imc}
          nombres={data.nombres}
      />
  );
;
```

```jsx
export const IMCGroupGraphCard = () => {
  const { data, isPending, error } = useGetList("infantes", {
      pagination: { page: 1, perPage: 12 },
      sort: { field: "id", order: "ASC" },
  });

  if (isPending) return <p>Cargando...</p>;
  if (error) return <p>Error al cargar los infantes</p>;

  /*Se consigue la informacion de todos los infantes
  y se guarda en un array de tarjetas para despues imprimirlas en la pagina*/
  const tarjetas = [];
  for (let i = 0; i < data.length; i++) {
      const kid = data[i];
      tarjetas.push(
          <IMCIndividualGraphCard key={kid.id} idInfante={kid.id} nombre={kid.nombr
      );
  }

  return (
      <Box sx={{ display: "flex", flexWrap: "wrap", gap: 4, mt: 2 }}>
          {tarjetas}
      </Box>
  );
```

Herramienta: Claude
Prompt: Como hago que las gráficas reciban los datos
Resultado:

```
const { data, isPending, error } = useGetList("mediciones", {
    pagination: { page: 1, perPage: 200 },
    sort: { field: "id", order: "ASC" },
});

if (isPending) return <p>Cargando...</p>;
if (error) return <p>Error al cargar las mediciones</p>;
/*Los datos se almacenan en un objeto por infante que funciona como un
diccionario. Utiliza el id_infante como llave y almacena adentro el nombre
del infante como id y un array de los datos del niño.*/
const porInfante = {};
data.forEach((m) => {
    if (!porInfante[m.id_infante]) {
        porInfante[m.id_infante] = { id: m.infante, data: [] };
    }
    porInfante[m.id_infante].data.push({ x: m.edad_meses, y: m.talla });
});
/*Se obtienen los valores dentro del objeto, removiendo la llave que era el
id_infante y se ordenan los datos que llegan desordenados para que en la graf
salgan de menor a mayor. Esta es una recomendacion de Claude
para que la grafica se vea mejor y no se vea desordenada.*/
const series = Object.values(porInfante).map((s) => ({
    ...s,
    data: s.data.sort((a, b) => a.x - b.x),
```

```
const { data, isPending, error } = useGetList("mediciones", {
    pagination: { page: 1, perPage: 200 },
    sort: { field: "id", order: "ASC" },
});

if (isPending) return <p>Cargando...</p>;
if (error) return <p>Error al cargar las mediciones</p>;

/*Los datos se almacenan en un objeto por infante que funciona como un
diccionario. Utiliza el id_infante como llave y almacena adentro el nombr
del infante como id y un array de los datos del niño.*/
const porInfante = {};
data.forEach((m) => {
    if (!porInfante[m.id_infante]) {
        porInfante[m.id_infante] = { id: m.infante, data: [] };
    }
    porInfante[m.id_infante].data.push({ x: m.talla, y: m.peso });
});
/*Se obtienen los valores dentro del objeto, removiendo la llave que era
id_infante y se ordenan los datos que llegan desordenados para que en la
salgan de menor a mayor. Esta es una recomendacion de Claude
para que la grafica se vea mejor y no se vea desordenada.*/
const series = Object.values(porInfante).map((s) => ({
    ...s,
```

Herramienta: Claude

Prompt: Con los datos que se encuentran en el excel a continuación {Inserte el excel de la página de la CDC con los percentiles de IMC} Genera una tabla en SQL para almacenar los percentiles de niños de 2 a 6 años e inyecta los datos correspondientes.

Resultado:

```
CREATE TABLE cdc_imc_percentiles (
    sexo CHAR(1) NOT NULL,
    edad_meses TINYINT UNSIGNED NOT NULL,
    p5  DECIMAL(5,2) NOT NULL,
    p85 DECIMAL(5,2) NOT NULL,
    p95 DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (sexo, edad_meses)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Le añadi el Engine y Charset de nuestras otras tablas

```
INSERT INTO cdc_imc_percentiles (sexo, edad_meses, p5, p85, p95) VALUES
('H',24,14.72,18.12,19.28),
('H',25,14.68,18.04,19.16),
('H',26,14.65,17.96,19.06),
('H',27,14.61,17.88,18.95),
('H',28,14.58,17.81,18.85),
('H',29,14.55,17.74,18.76),
('H',30,14.51,17.67,18.67),
('H',31,14.48,17.60,18.59),
('H',32,14.45,17.54,18.51),
('H',33,14.42,17.48,18.43),
('H',34,14.39,17.43,18.36),
('H',35,14.36,17.38,18.30),
('H',36,14.33,17.33,18.24),
('H',37,14.30,17.28,18.18),
('H',38,14.27,17.23,18.13),
('H',39,14.25,17.19,18.08),
('H',40,14.22,17.15,18.04),
('H',41,14.19,17.12,18.00),
('H',42,14.17,17.08,17.97),
('H',43,14.14,17.05,17.93),
('H',44,14.12,17.02,17.91),
('H',45,14.10,16.99,17.88),
('H',46,14.08,16.97,17.86),
('H',47,14.05,16.95,17.85),
```

```
('H',48,14.03,16.93,17.84),
('H',49,14.01,16.91,17.83),
('H',50,13.99,16.89,17.82),
('H',51,13.98,16.88,17.82),
('H',52,13.96,16.87,17.82),
('H',53,13.94,16.86,17.83),
('H',54,13.92,16.85,17.83),
('H',55,13.91,16.84,17.84),
('H',56,13.89,16.84,17.86),
('H',57,13.88,16.84,17.87),
('H',58,13.86,16.84,17.89),
('H',59,13.85,16.84,17.91),
('H',60,13.84,16.84,17.94),
('H',61,13.83,16.85,17.97),
('H',62,13.82,16.85,18.00),
('H',63,13.80,16.86,18.03),
('H',64,13.79,16.87,18.06),
('H',65,13.79,16.88,18.10),
('H',66,13.78,16.90,18.14),
('H',67,13.77,16.91,18.18),
('H',68,13.76,16.93,18.22),
('H',69,13.75,16.95,18.27),
('H',70,13.75,16.97,18.31),
('H',71,13.74,16.99,18.36),
('H',72,13.74,17.01,18.41),
('M',24,14.38,17.97,19.06),
```

```
('M',25,14.35,17.89,18.97),
('M',26,14.31,17.80,18.88),
('M',27,14.28,17.73,18.80),
('M',28,14.24,17.65,18.72),
('M',29,14.21,17.58,18.64),
('M',30,14.18,17.51,18.58),
('M',31,14.15,17.44,18.51),
('M',32,14.12,17.38,18.45),
('M',33,14.09,17.32,18.40),
('M',34,14.06,17.27,18.35),
('M',35,14.03,17.22,18.30),
('M',36,14.00,17.17,18.25),
('M',37,13.97,17.12,18.22),
('M',38,13.95,17.08,18.18),
('M',39,13.92,17.04,18.15),
('M',40,13.90,17.00,18.12),
('M',41,13.87,16.96,18.10),
('M',42,13.85,16.93,18.08),
('M',43,13.82,16.90,18.06),
('M',44,13.80,16.88,18.05),
('M',45,13.78,16.85,18.04),
('M',46,13.76,16.83,18.03),
('M',47,13.74,16.82,18.03),
('M',48,13.71,16.80,18.03),
('M',49,13.69,16.79,18.03),
('M',50,13.68,16.78,18.04),
```

```
('M',51,13.66,16.77,18.05),
('M',52,13.64,16.76,18.06),
('M',53,13.62,16.76,18.07),
('M',54,13.61,16.76,18.09),
('M',55,13.59,16.76,18.11),
('M',56,13.57,16.77,18.14),
('M',57,13.56,16.77,18.16),
('M',58,13.55,16.78,18.19),
('M',59,13.53,16.79,18.22),
('M',60,13.52,16.80,18.26),
('M',61,13.51,16.82,18.29),
('M',62,13.50,16.83,18.33),
('M',63,13.49,16.85,18.37),
('M',64,13.48,16.87,18.42),
('M',65,13.47,16.89,18.46),
('M',66,13.46,16.92,18.51),
('M',67,13.45,16.94,18.56),
('M',68,13.45,16.97,18.61),
('M',69,13.44,17.00,18.66),
('M',70,13.43,17.03,18.72),
('M',71,13.43,17.07,18.78),
('M',72,13.43,17.10,18.84);
```